

# SUJET DE TP : SAÉ 2.03 GESTION DE TICKETS 'UN PARC INFORMATIQUE

## Objectif du TP

L'objectif de ce TP est de mettre en pratique les concepts vus en cours magistral (PHP avec PDO, MySQL, Bootstrap) pour développer une application web fonctionnelle de gestion de tickets d'intervention pour un parc informatique. Ce TP est conçu pour être réalisé en 4 heures.

## Prérequis

- \* Un serveur web local (XAMPP, WAMP, MAMP ou équivalent) avec Apache et MySQL démarrés.
- \* Un éditeur de code (VS Code, Sublime Text, etc.).
- \* Les fichiers de base fournis (template Bootstrap, fichiers PHP).

## Étape 1 : Mise en place de la Base de Données

1. Ouvrez phpMyAdmin (accessible via `http://localhost/phpmyadmin`).
2. Créez une nouvelle base de données nommée `gestion\_tickets`.
3. Exécutez les scripts SQL suivants pour créer les tables nécessaires et insérer quelques données de test. Notez que le mot de passe de l'administrateur est déjà haché.
  - structure.sql
  - donnees.sql
4. Examinez attentivement les scripts pour comprendre ce qu'ils font (création des tables puis insertion des données). N'hésitez pas à poser des questions. (voir pdf gestion de la base de données)

## Étape 2 : Configuration et Connexion

### 2.1. Structure du dossier de TP

Créez un dossier (si ce n'est pas déjà fait) pour le TP avec l'arborescence suivante :

 app

 BDD

 index.php

Le dossier app contiendra nos fichiers php.

Déposez les deux fichiers .sql précédemment utilisés dans le dossier BDD

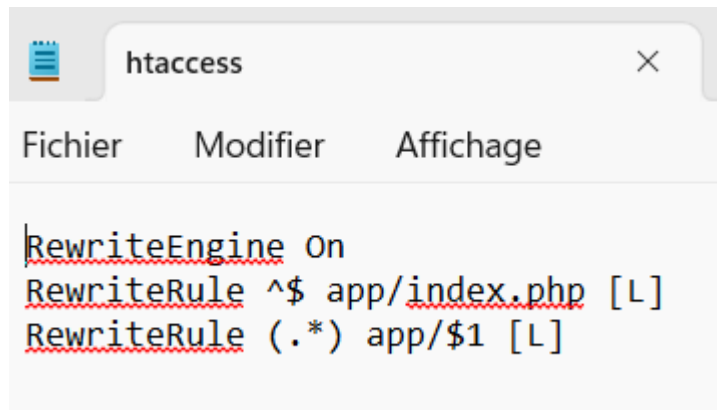
Le fichier index.php peut servir à la redirection (voir ci-après)

### 2.2. Configuration de la Redirection

Pour que notre projet pointe directement sur le dossier `app/` depuis la racine de notre serveur web, vous avez deux options :

### Option 1 (Recommandée - Serveur Apache) : Utilisation d'un fichier `.htaccess`

1. À la racine de votre dossier de TP (au même niveau que le dossier `app`), créez un fichier nommé `.htaccess`.
2. Ajoutez le contenu suivant à ce fichier :

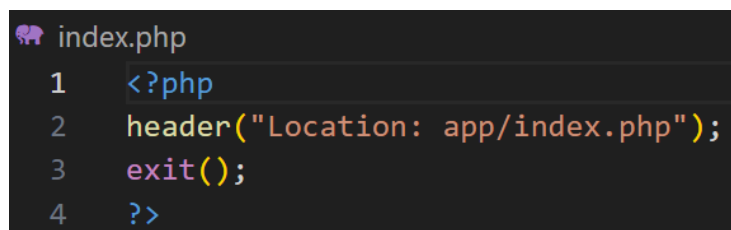


```
htaccess
Fichier  Modifier  Affichage
RewriteEngine On
RewriteRule ^$ app/index.php [L]
RewriteRule (.*) app/$1 [L]
```

Pour que cela fonctionne, le `mod_rewrite` doit être activé sur votre serveur Apache.

### Option 2 (Alternative présentée au-dessus) : Utilisation d'un fichier `index.php` de redirection

1. À la racine de votre dossier `sae203_tp` (au même niveau que le dossier `app`), créez un fichier nommé `index.php`.
2. Ajoutez le contenu suivant à ce fichier :

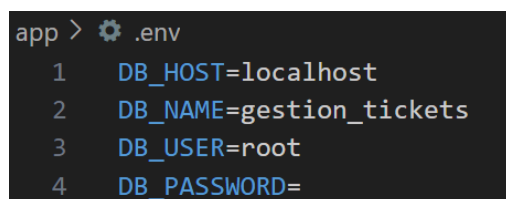


```
index.php
1 <?php
2 header("Location: app/index.php");
3 exit();
4 ?>
```

Cette méthode redirigera simplement le navigateur vers `app/index.php`.

### 2.3. Fichier `.env` pour la Base de Données

1. Dans le dossier `app`, créez un fichier `.env` et ajoutez-y les informations de connexion à votre base de données :



```
app > .env
1 DB_HOST=localhost
2 DB_NAME=gestion_tickets
3 DB_USER=root
4 DB_PASSWORD=
```

### 2.4 inclusions de fichiers

Dans le dossier `app`, créez un sous-dossier `includes`.

Dans `includes`, créez un fichier `config.php` avec le code suivant pour établir la connexion PDO en utilisant les informations du fichier `.env` :

app > includes > config.php > ...

```
1  <?php
2  // Chemin vers le fichier .env
3  $envFile = __DIR__ . "../.env";
4
5  // Vérification de l'existence du fichier .env
6  if (!file_exists($envFile)) {
7      die("Le fichier .env n'existe pas. Veuillez le créer à la racine de l'application.");
8  }
9
10 // Lecture du fichier .env et récupération des variables d'environnement
11 $env = parse_ini_file($envFile);
12
13 // Récupération des variables d'environnement
14 $dbHost = $env["DB_HOST"];
15 $dbName = $env["DB_NAME"];
16 $dbUser = $env["DB_USER"];
17 $dbPassword = $env["DB_PASSWORD"];
18
19 try {
20     // Connexion à la base de données avec PDO
21     $pdo = new PDO("mysql:host=$dbHost;dbname=$dbName;charset=utf8", $dbUser, $dbPassword);
22     // Définition des attributs de PDO pour afficher les erreurs
23     $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
24 } catch (PDOException $e) {
25     die("Erreur de connexion à la base de données : " . $e->getMessage());
26 }
27 ?>
```

Toujours dans `includes`, créez un fichier `functions.php` pour centraliser les fonctions d'interaction avec la base de données (CRUD) :

app > includes > functions.php > getUserByEmail

```
1  <?php
2
3  /**
4   * Récupère tous les services disponibles.
5   * @param PDO $pdo La connexion PDO à la base de données.
6   * @return array Un tableau d'objets représentant les services.
7   */
8  function getServices(PDO $pdo): array
9  {
10     $stmt = $pdo->query("SELECT id_service, nom_service FROM Service ORDER BY nom_service ASC");
11     return $stmt->fetchAll(PDO::FETCH_ASSOC);
12 }
13
14 /**
15 * Récupère un utilisateur par son email.
16 * @param PDO $pdo La connexion PDO à la base de données.
17 * @param string $email L'email de l'utilisateur.
18 * @return array|false Un tableau associatif représentant l'utilisateur ou false si non trouvé.
19 */
20 function getUserByEmail(PDO $pdo, string $email)
21 {
22     $stmt = $pdo->prepare("SELECT id_utilisateur, email_utilisateur, mot_de_passe,
23     role FROM Utilisateur WHERE email_utilisateur = ?");
24     $stmt->execute([$email]);
25     return $stmt->fetch(PDO::FETCH_ASSOC);
26 }
```

```

28  /**
29   * Crée un nouvel utilisateur.
30   * @param PDO $pdo La connexion PDO à la base de données.
31   * @param string $nom Le nom de l'utilisateur.
32   * @param string $prenom Le prénom de l'utilisateur.
33   * @param string $email L'email de l'utilisateur.
34   * @param string $password_hash Le mot de passe haché.
35   * @param string $role Le rôle de l'utilisateur (utilisateur ou administrateur).
36   * @return int L'ID du nouvel utilisateur inséré.
37   */
38  function createUser(PDO $pdo, string $nom, string $prenom, string $email,
39                      string $password_hash, string $role = 'utilisateur'): int
40  {
41      $stmt = $pdo->prepare("INSERT INTO Utilisateur (nom_utilisateur, prenom_utilisateur, email_utilisateur,
42                          mot_de_passe, role)
43                          VALUES (?, ?, ?, ?, ?)");
44      $stmt->execute([$nom, $prenom, $email, $password_hash, $role]);
45      return $pdo->lastInsertId();
46  }
47

```

```

48  /**
49   * Crée un nouveau ticket.
50   * @param PDO $pdo La connexion PDO à la base de données.
51   * @param int $id_utilisateur L'ID de l'utilisateur créant le ticket.
52   * @param int $id_service L'ID du service concerné.
53   * @param string $sujet Le sujet du ticket.
54   * @param string $description La description du ticket.
55   * @return int L'ID du nouveau ticket inséré.
56   */
57  function createTicket(PDO $pdo, int $id_utilisateur, int $id_service,
58                      string $sujet, string $description): int
59  {
60      $stmt = $pdo->prepare("INSERT INTO Ticket (id_utilisateur, id_service,
61                          sujet_ticket, description_ticket) VALUES (?, ?, ?, ?)");
62      $stmt->execute([$id_utilisateur, $id_service, $sujet, $description]);
63      return $pdo->lastInsertId();
64  }

```

```

66  /**
67   * Récupère les détails d'un ticket.
68   * @param PDO $pdo La connexion PDO à la base de données.
69   * @param int $id_ticket L'ID du ticket.
70   * @return array|false Un tableau associatif représentant le ticket ou false si non trouvé.
71   */
72  function getTicketDetails(PDO $pdo, int $id_ticket)
73  {
74      $sql = "SELECT t.id_ticket, t.sujet_ticket, t.description_ticket, t.date_creation,
75              t.statut_ticket, t.priorite_ticket, t.date_derniere_maj,
76              t.commentaire_admin, u.email_utilisateur, s.nom_service
77              FROM Ticket t
78              JOIN Utilisateur u ON t.id_utilisateur = u.id_utilisateur
79              JOIN Service s ON t.id_service = s.id_service
80              WHERE t.id_ticket = ?";
81      $stmt = $pdo->prepare($sql);
82      $stmt->execute([$id_ticket]);
83      return $stmt->fetch(PDO::FETCH_ASSOC);
84  }

```

```

86  /**
87   * Récupère tous les tickets pour l'affichage administrateur.
88   * @param PDO $pdo La connexion PDO à la base de données.
89   * @return array Un tableau d'objets représentant les tickets.
90   */
91  function getAllTickets(PDO $pdo): array
92  {
93      $sql = "SELECT t.id_ticket, t.sujet_ticket, t.description_ticket, t.date_creation,
94             t.statut_ticket, t.priorite_ticket, t.date_derniere_maj,
95             t.commentaire_admin, u.email_utilisateur, s.nom_service
96             FROM Ticket t
97             JOIN Utilisateur u ON t.id_utilisateur = u.id_utilisateur
98             JOIN Service s ON t.id_service = s.id_service
99             ORDER BY t.date_creation DESC";
100     $stmt = $pdo->query($sql);
101     return $stmt->fetchAll(PDO::FETCH_ASSOC);
102  }
103

```

```

104  /**
105   * Met à jour un ticket.
106   * @param PDO $pdo La connexion PDO à la base de données.
107   * @param string $statut Le nouveau statut du ticket.
108   * @param string $priorite La nouvelle priorité du ticket.
109   * @param string|null $commentaire_admin Le commentaire de l'administrateur.
110   * @param int $id_ticket L'ID du ticket à mettre à jour.
111   * @return bool Vrai si la mise à jour a réussi, faux sinon.
112   */
113  function updateTicket(PDO $pdo, string $statut, string $priorite, ?string $commentaire_admin,
114                        int $id_ticket): bool
115  {
116      $sql = "UPDATE Ticket SET statut_ticket = ?, priorite_ticket = ?, commentaire_admin = ?
117             WHERE id_ticket = ?";
118      $stmt = $pdo->prepare($sql);
119      return $stmt->execute([$statut, $priorite, $commentaire_admin, $id_ticket]);
120  }
121
122  ?>

```

## Étape 3 : Page d'Accueil et Ouverture de Ticket

Dans le dossier app, créez les sous-dossiers template et css.

Dans le dossier css, ajoutez le fichier fourni (style.css)

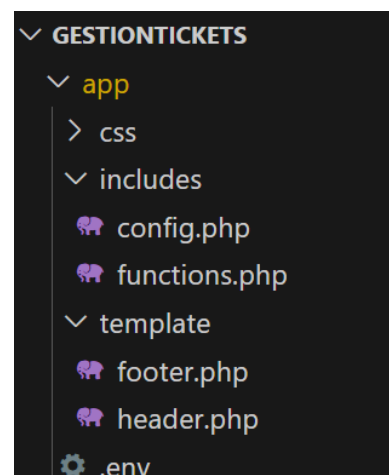
Dans le dossier template, insérez les fichiers header.php et footer.php pour structurer vos pages et éviter des redondances :

### Template : Header.php

Observez les lignes 6, 22, 24, 26, 31 et 33. Que permettent-elles de faire ?

### Template : Footer.php

À quoi sert le script intégré à la page ?



**NB : les fichiers sont désormais à inclure directement à la racine du dossier app**

**Intégrez le fichier index.php.**

Observez comment sont inclus les fichiers header.php et footer.php.

Comment sont intégrés les services récupérés à la ligne 9 dans le formulaire ?

**Intégrez le fichier submit\_ticket.php.**

Que font les fonctions **explode()** et **ucfirst()** en lignes 21 et 22 ?

**Ligne 29 : utilisation d'un paramètre ticket\_id (\$new\_ticket\_id)**

Observez la transmission du numéro de ticket : le paramètre est passé via une **requête GET**, directement intégrée dans l'URL de redirection.

C'est la méthode la plus simple pour transmettre une information d'une page à une autre après une redirection. Ici, cela permet à la page de suivi de savoir quel ticket elle doit afficher et de confirmer à l'utilisateur que l'opération a réussi.

**Attention** : puisque ces données apparaissent en clair dans la barre d'adresse, il ne faut jamais passer d'informations sensibles (comme un mot de passe) par ce canal. Pour un ID de ticket, c'est tout à fait approprié.

## Étape 4 : Suivi de Ticket

**Intégrez le fichier track\_ticket.php.**

Ce fichier doit récupérer l'ID du ticket passé en paramètre d'URL (**\$\_GET['ticket\_id']**). Il utilise la fonction **getTicketDetails(\$pdo, \$ticket\_id)** pour récupérer les informations du ticket, le nom du service et l'email de l'utilisateur.

Remarquez comment ces informations sont affichées de manière claire grâce à l'utilisation des composants Bootstrap (statut et priorité).

Pourquoi utilise-t-on **include\_once** aux lignes 3 et 4 ? Est-ce le plus approprié ?

Vous avez sans doute compris **que require\_once** serait bien plus adapté ici : ce sera également le cas pour les fichiers suivants :

- index.php
- login.php
- submit\_ticket.php
- update\_ticket.php
- admin\_dashboard.php

Les fichiers **template/header.php** et **template/footer.php** peuvent rester en **include\_once** car, bien que recommandés, leur absence n'empêche pas techniquement le script PHP de traiter les données (même si l'affichage sera cassé)

## Étape 5 : Espace Administrateur

**Intégrez la page `login.php` qui contient le formulaire de connexion (Email, Mot de passe).**

Pourquoi appelle-t-on la fonction **session\_start()** en ligne 2 ?

À quoi servent les fonctions **trim()**, **htmlspecialchars()** et **password\_verify()** ?

Observez le contenu de l'action du formulaire à la ligne 70. Où est-il traité ?

| Approche                                                        | Validité HTML                    | Sécurité                   | Simplicité                                           |
|-----------------------------------------------------------------|----------------------------------|----------------------------|------------------------------------------------------|
| action=""                                                       | <b>Non valide</b><br>(selon W3C) | Bonne                      | Très simple                                          |
| action="login.php"                                              | <b>Valide</b>                    | Bonne                      | Simple, mais rigide<br>(si vous renommez le fichier) |
| action="<?php echo htmlspecialchars(\$_SERVER["PHP_SELF"]); ?>" | <b>Valide</b>                    | Excellente<br>(si protégé) | Dynamique et robuste                                 |

**Intégrez le fichier admin\_dashboard.php :**

Comment est sécurisée cette page ?

Que fait la fonction getAllTickets(\$pdo) ?

Que remarquez-vous concernant la déclaration de variable dans le script JS ?

L'utilisation de **var** est aujourd'hui considérée comme une ancienne pratique (pré-2015, ES6).

Je vous invite donc à remplacer le mot clé **var** par **const**.

Le script devrait désormais ressembler à ceci :

```
<script>
const editTicketModal = document.getElementById('editTicketModal')
editTicketModal.addEventListener('show.bs.modal', function (event) {
    const button = event.relatedTarget
    const id_ticket = button.getAttribute('data-id')
    const sujet = button.getAttribute('data-sujet')
    const description = button.getAttribute('data-description')
    const statut = button.getAttribute('data-statut')
    const priorite = button.getAttribute('data-priorite')
    const commentaire = button.getAttribute('data-commentaire')

    const modal_id_ticket = editTicketModal.querySelector('#modal_id_ticket')
    const modal_sujet = editTicketModal.querySelector('#modal_sujet')
    const modal_description = editTicketModal.querySelector('#modal_description')
    const modal_statut = editTicketModal.querySelector('#modal_statut')
    const modal_priorite = editTicketModal.querySelector('#modal_priorite')
    const modal_commentaire_admin = editTicketModal.querySelector('#modal_commentaire_admin')

    modal_id_ticket.value = id_ticket
    modal_sujet.value = sujet
    modal_description.value = description
    modal_statut.value = statut
    modal_priorite.value = priorite
    modal_commentaire_admin.value = commentaire
})
</script>
```

**Petit conseil supplémentaire :** N'hésitez pas à ajouter des points-virgules (;) à la fin de vos lignes JS.

Bien qu'ils soient souvent optionnels, ils évitent des erreurs de compression ou de lecture par le navigateur dans certains cas complexes.

### Intégrez update\_ticket.php

Ce fichier traite la mise à jour en base de données en utilisant la fonction **updateTicket(\$pdo, ...)**.

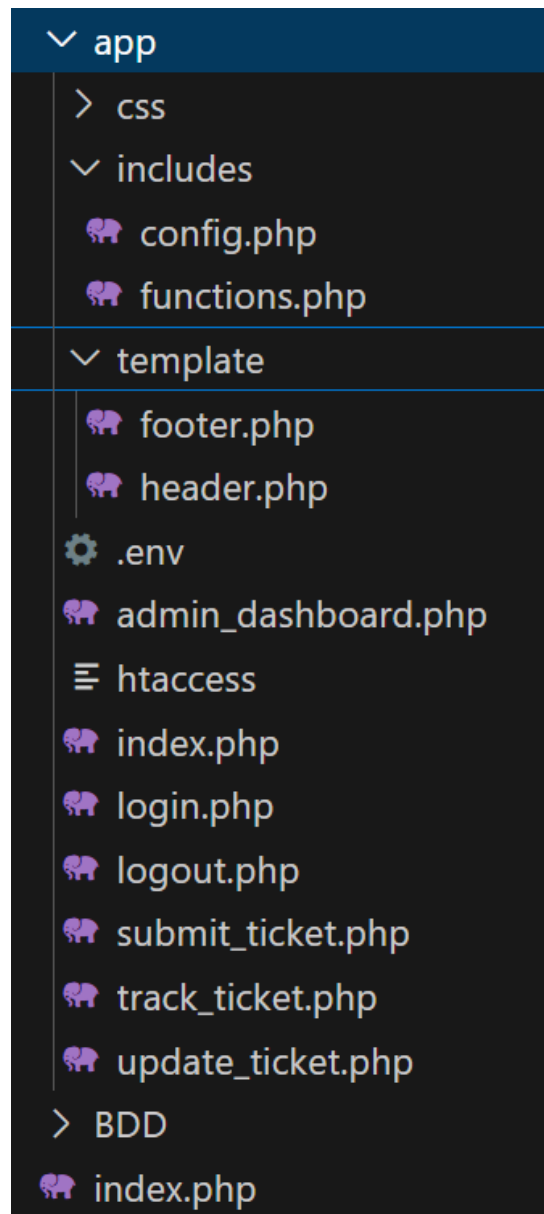
Retournez voir son fonctionnement et notamment la requête d'insertion (requête préparée) dans le fichier **fonctions.php**.

Une fois de retour sur update\_ticket.php, remplacez les **include\_once** par **require\_once** si vous ne l'avez pas encore fait.

### Intégrez le fichier logout.php :

Observez l'utilisation de la fonction exit() en fin de fichier. En quoi est-ce une bonne pratique en termes de sécurité et d'éco conception ?

## VOUS DEVRIEZ DÉSORMAIS DISPOSER DE L'ARBORESCENCE SUIVANTE



Prenez en main votre application : ajouter des tickets et assurez-en le suivi.

**Des améliorations** seront ajoutées dans une version 2 de cet exercice : en effet, l'accès au suivi d'un ticket n'est pas vraiment sécurisé n'est-ce pas ?

La suite au prochain TP ...